Guia de Estudo

Aplicação Java do Sistema do Bar

Projeto de Banco de Dados · Entrega 06 (versão final) · Spring Boot + PostgreSQL

Divisão da apresentação

	Pessoa 1: Estrutura, CRUD e Rotinas	Pessoa 2: Consultas e Dashboard
Assuntos	Visão geral e arquitetura, conceitos de Spring, CRUD das 9 tabelas, transações, tratamento de erros, integração com funções, procedimentos e triggers	Consultas e visões com filtros, extrator genérico, dashboard estatístico, teste t (p-valor), gráficos e testes automatizados
Arquivos	BarApplication, *Controller de cadastro, AtendimentoController, RotinaController, ErrosBanco, ConexaoAdvice, Cliente/Funcionario/Atendimento/RotinaRepository, model/*, application.properties	ConsultaController, TabelaController, DashboardController, ConsultaRepository, TabelaRepository, DashboardRepository, Resultados, TesteT, graficos.js, dashboard.js, consultas.js, testes
Demonstração	Abrir e fechar uma conta; trigger barrando pedido em conta fechada; reajuste de preço com log	Dashboard com filtros; uma consulta com filtro; uma visão; rodar os testes
Tempo sugerido	~ 6 a 7 min	~ 6 a 7 min

Como usar este guia

A **Parte Comum** (seções 1 e 2) os dois precisam saber: se o professor fizer uma pergunta geral para qualquer um, ambos devem conseguir responder.

Cada parte termina com **perguntas prováveis**. Treine respondendo em voz alta sem olhar.

O que mudou da Entrega 03 para a 06

CRUD passou de 3 para 9 tabelas (novas telas de Funcionários e Atendimentos); tela nova de Rotinas do banco (Etapa 05); consultas e visões com filtros, indicadores e gráfico; dashboard com filtros de período, categoria e área, 9 gráficos e mais medidas (moda, variância, assimetria); carga de histórico de julho a setembro; 121 testes automatizados com 100% de cobertura.

Parte Comum: os dois precisam saber

1. O que é a aplicação

É uma aplicação **web** feita em **Spring Boot 3.5** com **Java 17**, que se conecta ao banco **PostgreSQL** criado nas entregas anteriores. Ela tem estas áreas:

- **Dashboard**: indicadores, estatística descritiva e 9 gráficos, com filtros.
- **Consultas**: as 9 consultas das Etapas 03 e 04 e as 2 visões, com filtros, indicadores, gráfico e o SQL executado.
- **Atendimentos**: a conta da mesa, com pedidos e pagamentos parciais.
- **Rotinas do banco**: executa as funções, procedimentos e triggers da Etapa 05.
- **Cadastros**: Produtos, Mesas, Clientes e Funcionários.
- **Tabelas**: conteúdo das 10 tabelas e das 2 visões.

Frase de abertura sugerida:

> *"Nossa aplicação é um sistema web em Spring Boot sobre o banco PostgreSQL que modelamos. Todo o SQL foi escrito à mão com JdbcTemplate, sem ORM, e as regras ficam no banco: constraints, funções, procedimentos e triggers. A interface usa e mostra tudo isso."*

Tecnologias (pom.xml)

Dependência	Para que serve
spring-boot-starter-web	Servidor web embutido (Tomcat) e os controllers que respondem às URLs
spring-boot-starter-thymeleaf	Motor de templates: gera o HTML no servidor com os dados do banco
spring-boot-starter-jdbc	JdbcTemplate e NamedParameterJdbcTemplate (executam SQL) e o pool HikariCP
postgresql	Driver JDBC do PostgreSQL
spring-boot-starter-test + JaCoCo	Testes (JUnit 5, MockMvc) e relatório de cobertura
Chart.js 4.4.1 (static/js/vendor)	Biblioteca JavaScript dos gráficos, dentro do projeto (funciona sem internet)

Por que JDBC e não JPA/Hibernate?

Porque é uma disciplina de banco de dados. Com JPA o SQL é gerado automaticamente e fica escondido. Com **JdbcTemplate** cada SELECT, INSERT, UPDATE, DELETE e CALL está escrito no código, então dá para mostrar JOIN, CTE, funções de janela, funções de agregação e a chamada das rotinas.

2. Arquitetura em camadas (MVC)

```
Navegador --HTTP--> Controller --> Repository --SQL--> PostgreSQL
                        |               |               (funções, procedimentos, triggers)
                        |          Model (records)
                        v
          Template Thymeleaf (HTML) --> Navegador --> graficos.js desenha os gráficos
```

Pacote	Responsabilidade
controller	Recebe a requisição HTTP, chama o repositório, coloca os dados no Model e escolhe a página
repository	Somente acesso ao banco: cada método executa um SQL
model	Records com os dados que circulam entre as camadas (Atendimento, Funcionario, Consulta, Grafico...)
estatistica	TesteT: cálculo do p-valor que o PostgreSQL não faz
resources/templates	Páginas HTML (Thymeleaf)
resources/static	CSS (modo claro e escuro) e JavaScript dos gráficos

O **BarApplication** só tem o `main`. A anotação `@SpringBootApplication` faz o Spring procurar as classes com `@Controller`, `@Repository` e `@Component`, criar uma instância de cada e ligá-las entre si.

Roteiro da apresentação (ordem)

#	Quem	O quê
1	Pessoa 1	Abertura: o que é a aplicação, tecnologias e arquitetura
2	Pessoa 1	CRUD (Funcionários e Atendimentos), transações e erros + demo
3	Pessoa 1	Rotinas da Etapa 05 na interface + demo do trigger e do reajuste
4	Pessoa 1 → 2	Transição: "Além de gravar, a aplicação consulta e analisa os dados..."
5	Pessoa 2	Consultas e visões com filtros + demo
6	Pessoa 2	Dashboard: estatística no SQL, p-valor no Java, gráficos + demo dos filtros
7	Pessoa 2	Testes automatizados e fechamento

Pessoa 1: Estrutura, CRUD e Rotinas

3. Conceitos de Java/Spring usados no projeto

Injeção de dependência pelo construtor

Ninguém escreve `new AtendimentoRepository()`. O Spring cria o objeto e entrega pelo construtor. Os repositórios recebem o `JdbcTemplate` do mesmo jeito, já configurado com os dados do application.properties.

```
@Controller
public class AtendimentoController {
    private final AtendimentoRepository atendimentos;

    public AtendimentoController(AtendimentoRepository atendimentos) {   // o Spring injeta
        this.atendimentos = atendimentos;
    }
```

Records (pacote model)

São classes de dados **imutáveis** (Java 16+). O Java gera o construtor, os getters (`a.status()`), o equals, o hashCode e o toString. Um record pode ter método próprio:

```
public record Atendimento(int id, LocalDateTime abertura, String status, ..., BigDecimal consumido,
                          BigDecimal pago, String situacao) {
    public BigDecimal saldo() { return consumido.subtract(pago); }
}
```

RowMapper: de linha do banco para objeto

É uma lambda que o JdbcTemplate chama para **cada linha** do ResultSet:

```
private static final RowMapper<Mesa> MAPPER = (rs, i) -> new Mesa(
        rs.getInt("Numero_Mesa"), rs.getInt("Capacidade"), rs.getString("Localizacao"));
```

Parâmetros ? e SQL injection

Todo valor vindo do usuário vai como parâmetro `?` (ou `:nome`, nos filtros). O JdbcTemplate usa **PreparedStatement**: o SQL e os valores vão separados para o banco, então um texto malicioso nunca é executado como SQL.

4. Fluxo do CRUD (igual em todas as telas de cadastro)

Ação	URL	Método do controller	O que faz
Listar	GET /funcionarios	lista()	Põe no Model a lista e um formulário vazio (com a próxima matrícula)
Abrir edição	GET /funcionarios?editar=CPF	lista()	Formulário vem preenchido e edicao = true
Filtrar	GET /funcionarios?cargo=Gerente	lista()	Lista só o cargo escolhido
Salvar	POST /funcionarios/salvar	salvar()	Se edicao: atualizar(); senão: inserir()
Excluir	POST /funcionarios/{cpf}/excluir	excluir()	Chama excluir() (o HTML pede confirm())

Post/Redirect/Get + Flash Attributes

Depois de um POST, o controller **não** devolve a página diretamente. Ele retorna `redirect:/...` e a mensagem de sucesso ou erro vai em `addFlashAttribute`, que sobrevive a um redirect. Assim, se o usuário der F5, o formulário não é reenviado.

Tabelas com CRUD pela interface

Tela	Tabelas	Destaque
Produtos	PRODUTO	Mudar o preço dispara o trigger de log
Mesas	MESA	CHECK de capacidade > 0
Clientes	PESSOA, CLIENTE, TELEFONE	Especialização + atributo multivalorado numa transação
Funcionários	PESSOA, FUNCIONARIO, TELEFONE	Supervisor (autorrelacionamento) e ON DELETE SET NULL
Atendimentos	ATENDIMENTO, PEDIDO, PAGAMENTO_PARCIAL	Usa fn_valor_conta e fn_situacao_conta; trigger de conta fechada

5. Cliente e Funcionário: especialização refletida no código

> **Por que Cliente e Funcionário mexem em 3 tabelas**
>
> **Especialização:** um cliente ou funcionário é uma linha em **PESSOA** + uma linha em **CLIENTE** ou **FUNCIONARIO** com o mesmo CPF.
>
> **Atributo multivalorado:** os telefones ficam na tabela **TELEFONE** (CPF, Telefone).
>
> Por isso cada operação precisa ser uma **transação** (`@Transactional`).

```
@Transactional
public void inserir(Funcionario f) {
    jdbc.update("INSERT INTO PESSOA (CPF, Nome, Rua, Numero, Bairro, CEP) VALUES (...)", ...);
    jdbc.update("INSERT INTO FUNCIONARIO (CPF, Matricula, Cargo, Salario, CPF_Supervisor) VALUES (...)", ...);
    inserirTelefones(f.cpf(), f.telefones());
}
```

- **@Transactional** = **atomicidade** (o A do ACID): se o INSERT em FUNCIONARIO falhar (ex.: matrícula repetida), o INSERT em PESSOA é desfeito.
- Na atualização, os telefones são substituídos: apaga todos e insere a lista nova.
- `inserirTelefones` separa a lista por **vírgula ou ponto e vírgula**, tira o que não é dígito (então "(81) 99999-0000" vira 81999990000), ignora vazios e repetidos e faz um INSERT por telefone.
- A listagem usa **JOIN** com PESSOA, **LEFT JOIN** com a PESSOA do supervisor (quem não tem supervisor também aparece) e **STRING_AGG** para juntar os telefones.

> **ON DELETE SET NULL na prática**
>
> A FK de supervisor em FUNCIONARIO é `ON DELETE SET NULL`. Ao excluir um gerente, a equipe dele fica sem supervisor e a tela avisa quantos ficaram assim, sugerindo o procedimento de redistribuição. Quem já atendeu mesa não pode ser excluído: a FK de ATENDIMENTO barra.

6. Atendimentos: a conta da mesa

Ação	SQL / regra
Listar contas	`fn_valor_conta(id)` e `fn_situacao_conta(id)` no SELECT; filtro por status, situação e período
Abrir conta	INSERT em ATENDIMENTO com status ABERTA
Lançar / alterar item	INSERT / UPDATE em PEDIDO (passam pelo trigger `trg_pedido_conta_fechada`)
Registrar pagamento	INSERT em PAGAMENTO_PARCIAL com `Num_Transacao = MAX + 1`; CHECK do método
Fechar conta	Só fecha se `fn_situacao_conta` devolver QUITADA, PAGO A MAIS ou SEM CONSUMO
Remover item / pagamento	DELETE com `AND EXISTS (... Status_Conta = 'ABERTA')`: em conta fechada não remove nada
Excluir atendimento	Transação: apaga PEDIDO e ATENDIMENTO; PAGAMENTO_PARCIAL sai por ON DELETE CASCADE

> **Por que a remoção é barrada na aplicação**
>
> O trigger cobre INSERT e UPDATE em PEDIDO. Um DELETE em conta fechada também mudaria o total de uma conta já paga, então a aplicação só apaga quando a conta está ABERTA (a condição está no próprio DELETE).

7. Rotinas da Etapa 05 na interface (tela Rotinas do banco)

Rotina	Como a tela usa / mostra
fn_valor_conta fn_situacao_conta	Formulário com o nº do atendimento mostra os dois resultados; gráfico de rosca com as contas por situação e tabela das contas não quitadas. Também usadas em Atendimentos e no dashboard.
sp_reajustar_preco_categoria	`CALL` com categoria e percentual. As mensagens `RAISE NOTICE` aparecem na tela; percentual <= -100 ou categoria sem produto dá a exceção do procedimento.
trg_log_preco_produto	Tabela LOG_PRECO_PRODUTO com preço antigo, novo, variação, data e usuário.
sp_redistribuir_sem_supervisor	Botão que simula a saída de um gerente (equipe fica sem supervisor) e botão que executa o procedimento (cursor). Gráfico de supervisionados por gerente antes e depois.
trg_pedido_conta_fechada	Teste de INSERT em PEDIDO numa transação que sempre faz ROLLBACK: conta aberta aceita, conta fechada mostra a exceção do trigger. Nada é gravado.

Como as mensagens RAISE NOTICE chegam ao Java

```
String sql = "CALL sp_reajustar_preco_categoria(CAST(? AS VARCHAR), CAST(? AS NUMERIC))";
try (PreparedStatement ps = con.prepareStatement(sql)) {
    ...
    ps.execute();
    for (SQLWarning w = ps.getWarnings(); w != null; w = w.getNextWarning()) {
        avisos.add(w.getMessage());     // cada NOTICE vira um SQLWarning no driver JDBC
    }
}
```

- O código de cada rotina aparece na tela, lido do próprio banco com `pg_get_functiondef`.

- O teste do trigger usa um `TransactionTemplate` com `REQUIRES_NEW` e `setRollbackOnly()`: transação própria, sempre desfeita.

8. Tratamento de erros

ErrosBanco: traduz o SQLSTATE do PostgreSQL

SQLSTATE	Significado	Exemplo no sistema
23505	unique_violation: chave duplicada	CPF, matrícula, nome de produto ou nº de atendimento repetido
23503	foreign_key_violation	Excluir produto já pedido ou funcionário que já atendeu
23514	check_violation	Preço, capacidade ou salário <= 0; forma de pagamento inválida
23502	not_null_violation	Campo obrigatório vazio
22001	string_data_right_truncation	Texto maior que o VARCHAR da coluna
P0001	raise_exception (PL/pgSQL)	"A conta do atendimento 1 ja esta fechada", "Percentual invalido"

O ErrosBanco também tira o prefixo `ERROR:` e as linhas de contexto (`Where: PL/pgSQL function...`) para a mensagem ficar limpa.

> **Frase forte para a apresentação**
>
> As **regras ficam no banco** (PK, FK, CHECK, NOT NULL, triggers). O Java não duplica essas validações, só mostra a mensagem. Assim o banco garante a consistência mesmo que alguém acesse por outro programa.

ConexaoAdvice

Um `@ControllerAdvice` com `@ExceptionHandler(CannotGetJdbcConnectionException.class)`. Se o PostgreSQL estiver desligado, qualquer página mostra *erro-conexao.html* com instruções, em vez de uma stack trace.

9. Demonstração da Pessoa 1

- Em **Atendimentos**, abrir uma conta, lançar 2 itens e mostrar o valor calculado por fn_valor_conta.
- Tentar **fechar sem pagar**: a tela mostra que fn_situacao_conta devolveu EM ABERTO.
- Registrar o pagamento (o valor já vem preenchido com o saldo) e **fechar**.
- Tentar lançar outro item na conta fechada: aparece a exceção do **trigger**.
- Em **Rotinas do banco**, reajustar Cerveja em 10%: mostrar as mensagens NOTICE e as 5 linhas novas no log.
- (Opcional) Simular a saída de um gerente e executar a redistribuição (cursor).
- (Opcional) Em **Funcionários**, tentar excluir quem já atendeu mesa: erro de FK 23503.

10. Perguntas prováveis da Pessoa 1

Pergunta	Resposta
Onde está o SQL?	Nos *Repository, como text blocks (""""), executado pelo JdbcTemplate.
Como vocês evitam SQL injection?	Todos os valores vão como parâmetros (PreparedStatement).

Pergunta	Resposta
Para que serve o @Transactional?	Atomicidade: PESSOA, FUNCIONARIO e TELEFONE gravam juntos ou nada.
Como a interface usa as funções?	Chamando fn_valor_conta e fn_situacao_conta dentro dos SELECTs e na regra de fechar a conta.
Como mostram o efeito do trigger?	Lançando item em conta fechada (erro P0001) e pela tabela LOG_PRECO_PRODUTO.
Como o Java recebe o RAISE NOTICE?	O driver JDBC transforma cada NOTICE em SQLWarning no statement.
Por que o procedimento usa cursor?	Cada funcionário vai para o gerente com menos supervisionados naquele momento; a escolha depende das atribuições anteriores.
O teste do trigger grava dados?	Não: roda numa transação própria marcada para ROLLBACK.
Por que redirect depois do POST?	Padrão Post/Redirect/Get: evita reenviar o formulário no F5.
E se dois usuários pegarem o mesmo MAX+1?	A PK barra o segundo e ele recebe a mensagem de chave duplicada. Uma SEQUENCE resolveria.

Pessoa 2: Consultas, Dashboard e Testes

11. Consultas e visões com filtros

Cada consulta é um record `Consulta` com id, grupo, título, descrição, recursos SQL, a lista de **filtros** (`Parametro`), a configuração do **gráfico** (`Grafico`) e o SQL. O SQL sem filtros continua em *sql/consultas/*.

```
WHERE (CAST(:inicio AS DATE) IS NULL OR CAST(a.Data_Hora_Abertura AS DATE) >= CAST(:inicio AS DATE))
  AND (CAST(:area AS VARCHAR) IS NULL OR m.Localizacao = :area)
```

- Os filtros são **parâmetros nomeados** (`NamedParameterJdbcTemplate`). Campo vazio vira NULL e a condição deixa de filtrar, então o mesmo SQL serve com e sem filtro.
- Nas **mesas ociosas**, o período fica na condição do LEFT JOIN, não no WHERE: no WHERE ele transformaria o anti join num INNER JOIN.
- As opções das listas (áreas, categorias, cargos, garçons) vêm do banco com SELECT DISTINCT.
- A página mostra indicadores (linhas, total, média, maior valor ou moda), o gráfico e o SQL executado.

Consulta / visão	Recursos SQL	Filtros e gráfico
1. Faturamento por categoria	JOIN, GROUP BY, SUM(SUM()) OVER ()	período, área · rosca
2. Ranking de atendentes	JOIN de 6 tabelas, RANK() OVER	período, cargo · barras
3. Conferência de contas	2 CTEs, LEFT JOIN, CASE	período, status, situação · rosca
4. Clientes acima da média	CTE, subconsulta com AVG, correlacionada	período, fidelidade · barras
5. Hierarquia de supervisão	Autojunção, STRING_AGG	cargo · barras
6. Mesas ociosas	Anti join, COUNT() OVER	período, área · barras
7. Dia A x dia B	FULL OUTER JOIN, COALESCE	os dois dias, categoria · barras agrupadas
8. Clientes de todas as categorias	NOT EXISTS duplo (divisão)	fidelidade, mín. de visitas · barras
9. Produtos acima da média	Subconsultas correlacionadas	categoria · barras agrupadas
vw_itens_vendidos	Visão com JOIN de 6 tabelas	período, categoria, área, garçom, status · soma por categoria
vw_historico_cliente	Visão com LEFT JOIN e CASE	segmento, fidelidade, sugestão · contagem por segmento

> **Detalhe que pode ser perguntado**
>
> O valor de cada item é `Quantidade * Preco_Atual`, porque a tabela PEDIDO não guarda o preço cobrado no momento da venda. Por isso existe o log de preços.

12. Resultados.EXTRATOR e a tela de tabelas

Cada consulta retorna colunas diferentes, então um `ResultSetExtractor` genérico lê o **ResultSetMetaData** (nome e tipo das colunas) e monta um `ResultadoTabela(colunas, numericas, linhas)`. Um único fragmento HTML desenha qualquer resultado, com decimais no padrão brasileiro (15.658,00).

A tela **Tabelas** mostra as 10 tabelas (incluindo LOG_PRECO_PRODUTO) e as 2 visões. O nome vem da URL, mas só serve de chave num mapa de SELECTs fixos (**whitelist**): nada é concatenado no SQL e um nome fora da lista dá 404.

13. Dashboard (DashboardController + DashboardRepository)

Tudo parte da visão `vw_itens_vendidos` filtrada por **período**, **categoria** e **área**. Uma CTE `contas` agrupa os itens filtrados por atendimento; cada método do repositório é um SELECT sobre ela.

```
WITH itens AS (SELECT v.* FROM vw_itens_vendidos v WHERE ...filtros...),
     contas AS (SELECT ID_Atendimento, Data_Atendimento, ..., SUM(Valor_Item) AS total
                  FROM itens GROUP BY ...)
```

Indicadores

Faturamento e média por noite, atendimentos e contas abertas, ticket médio e itens por conta, itens vendidos e produto mais vendido, clientes atendidos e % com fidelidade, **tendência semanal** (variação da última semana e inclinação da reta com `REGR_SLOPE`), **% de contas quitadas** (usa `fn_situacao_conta`) e categoria líder.

Os 9 gráficos

Gráfico	Tipo	SQL / análise
Evolução semanal	Linha + tendência	DATE_TRUNC('week'); reta de mínimos quadrados; botões trocam a métrica
Valor por atendimento	Histograma + tabela	GENERATE_SERIES, LEFT JOIN, SUM(COUNT) OVER (ORDER BY)
Faturamento por categoria	Rosca	GROUP BY Categoria
Forma de pagamento	Pizza	PAGAMENTO_PARCIAL das contas do filtro
Ticket por dia da semana	Barras agrupadas	AVG x PERCENTILE_CONT(0.5) por ISODOW
Perfil de consumo por área	Radar	% por categoria com SUM(SUM()) OVER (PARTITION BY área)
10 produtos mais vendidos	Barras horizontais	ORDER BY ... LIMIT 10
Itens por hora	Barras	EXTRACT(HOUR); ordem começando às 12h para a noite passar da meia-noite
Permanência x valor	Dispersão + reta	CORR, REGR_SLOPE, REGR_INTERCEPT

Estatística descritiva (no PostgreSQL)

```
SELECT ROUND(AVG(total), 2)                                    AS media,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total)      AS mediana,
       MODE() WITHIN GROUP (ORDER BY total)                    AS moda,
       ROUND(VAR_SAMP(total), 2)                               AS variancia,
       ROUND(STDDEV_SAMP(total), 2)                            AS desvio_padrao,
       ROUND(100 * STDDEV_SAMP(total) / NULLIF(AVG(total), 0), 1) AS coef_variacao,
       3 * (AVG(total) - mediana) / STDDEV_SAMP(total)         AS assimetria, ... q1, q3, MIN, MAX
  FROM fechadas
```

- **VAR_SAMP / STDDEV_SAMP**: variância e desvio padrão amostrais (dividem por n - 1). A variância fica em R$².
- **Moda**: o valor exato mais repetido (MODE). **Classe modal**: a faixa do histograma com maior frequência.
- **Assimetria de Pearson** positiva: algumas contas altas puxam a média para cima (média > mediana).

- **NULLIF(AVG, 0)**: evita divisão por zero quando o filtro não tem dados.

Correlação de Pearson: tempo de permanência x valor gasto

- **Permanência** = da abertura da conta até o último pagamento, em minutos (`EXTRACT(EPOCH FROM ...) / 60`).
- **r** = `CORR(total, minutos)`; **r²** = coeficiente de determinação.
- Reta: `REGR_SLOPE` (inclinação) e `REGR_INTERCEPT` (intercepto).
- Estatística de teste: **t = r·√(n−2) / √(1−r²)**, com **gl = n − 2**.
- Com 1 ou 2 contas no filtro não há teste (r ou t ficam indefinidos) e a tela avisa.

14. TesteT: o único cálculo estatístico feito em Java

> **Por que existe**
>
> O PostgreSQL calcula r e t, mas **não tem a distribuição t de Student**. O p-valor é calculado em Java para decidir se a correlação é significativa.

Hipóteses: **H0: ρ = 0** x **H1: ρ ≠ 0** (bilateral). Se **p < 0,05**, rejeita-se H0.

```
p = I_x(gl/2, 1/2)      com   x = gl / (gl + t²)
```

- **I_x(a, b)** é a função beta incompleta regularizada, calculada por fração contínua (Numerical Recipes): itera até a diferença ficar menor que 10^{-15} ou atingir 300 iterações.
- O método `naoZero` troca valores perto de zero por 10^{-300} para evitar divisão por zero.
- **ln Γ(x)** usa a aproximação de Lanczos. Casos especiais: gl ≤ 0 ou t = NaN dá NaN; t infinito dá p = 0.

15. Do Java para os gráficos

```
<!-- Chart.js dentro do projeto e as funções comuns (barras, linhas, pizza, radar) -->
<script th:src="@{/js/vendor/chart.umd.min.js}"></script>
<script th:src="@{/js/graficos.js}"></script>
<script th:inline="javascript">
    window.DADOS = { semanal: /*[[${semanal}]]*/ [], ... };       // o Thymeleaf troca por JSON
</script>
```

Com `th:inline="javascript"`, o Thymeleaf converte as listas e mapas Java em JSON. O *graficos.js* lê as cores do CSS (funciona no modo claro e no escuro) e usa uma paleta de 8 cores pensada para daltonismo.

16. Testes automatizados

Tipo	O que cobre
Integração (MockMvc + PostgreSQL)	Todas as rotas, filtros, erros do banco (23505, 23503, 23514, P0001) e o efeito das rotinas. Cada teste roda numa transação desfeita no final; sem banco no ar, são pulados.
Unitários	Formatação, tradução de erros, p-valor do teste t e modelos.
Simulados (Mockito)	Queda de conexão e falhas que o banco real não provoca sob demanda.

121 testes, cobertura de **100%** das linhas e ramos (JaCoCo). Comando: `.\mvnw.cmd test` na pasta app. Os testes acharam bugs reais antes da entrega, como o gráfico das visões e a separação dos telefones por espaço.

17. Demonstração da Pessoa 2

- Abrir o **Dashboard**: indicadores, evolução semanal com tendência (trocar a métrica nos botões).
- Aplicar um **filtro** (ex.: categoria Chopp, área Area externa) e mostrar que tudo recalcula.
- Mostrar a estatística descritiva, o histograma e a dispersão com r, r², t e p-valor.
- Em **Consultas**, abrir Mesas ociosas e mudar o período; abrir uma visão e filtrar.
- (Opcional) Rodar `.\mvnw.cmd test` e abrir o relatório de cobertura.

18. Perguntas prováveis da Pessoa 2

Pergunta	Resposta
Onde é calculada a estatística?	Quase tudo no PostgreSQL (AVG, PERCENTILE_CONT, MODE, VAR_SAMP, STDDEV_SAMP, CORR, REGR_*). Só o p-valor é em Java.
Os dados do dashboard são do banco?	Sim: cada carregamento faz os SELECTs na hora, a partir da visão vw_itens_vendidos.
Como funciona o filtro sem montar SQL na mão?	Parâmetros nomeados: vazio vira NULL e a condição (:p IS NULL OR ...) deixa de filtrar.
De onde vieram os dados de julho e agosto?	Do sql/15_carga_historico.sql: carga determinística (setseed) que respeita as regras e o trigger.
O que significa p < 0,05?	A chance de observar essa correlação se não houvesse relação é menor que 5%, então rejeitamos H0.
Diferença entre média, mediana e moda?	Média sofre com extremos; mediana é o valor do meio; moda é o mais frequente.
O que é a tendência semanal?	A reta de mínimos quadrados sobre o faturamento de cada semana; a inclinação é o quanto cresce por semana.
O nome da tabela vem da URL. Dá SQL injection?	Não. É só chave de um mapa de SELECTs fixos (whitelist).
O que é uma função de janela?	Calcula sobre um conjunto de linhas sem agrupá-las, ex.: RANK() OVER, SUM(...) OVER ().
Como garantem que nada quebrou?	121 testes automatizados com 100% de cobertura, rodando contra o banco real.

Revisão final: os dois

19. Mapa rápido dos arquivos

Arquivo	Uma linha	Quem
BarApplication	main + @SpringBootApplication	P1
Produto/Mesa/Cliente/FuncionarioController	CRUD: lista, salvar, excluir, com PRG e flash	P1
Cliente/FuncionarioRepository	PESSOA + especialização + TELEFONE em transação	P1
AtendimentoController/Repository	Conta, pedidos, pagamentos; usa as funções e o trigger	P1
RotinaController/Repository	CALL dos procedimentos, NOTICE, log e teste do trigger	P1
ErrosBanco / ConexaoAdvice	SQLSTATE → mensagem; página se o banco estiver fora	P1
Formato	Números, moeda e datas no padrão brasileiro nos templates	P1
ConsultaController/Repository	9 consultas e 2 visões com filtros e gráfico	P2
TabelaController/Repository	10 tabelas e 2 visões por whitelist	P2
Resultados	Extrator genérico via ResultSetMetaData	P2
DashboardController/Repository	Indicadores, estatística e séries dos gráficos com filtros	P2
TesteT	p-valor da t de Student (beta incompleta)	P2
graficos.js / dashboard.js / consultas.js	Desenho dos gráficos com o Chart.js	P2
src/test/java	121 testes, 100% de cobertura	P2

20. Checklist antes de apresentar

- [ ] PostgreSQL rodando e banco **bar** recriado (scripts/rodar_tudo.ps1): 306 atendimentos.
- [ ] Aplicação no ar (`.\mvnw.cmd spring-boot:run` na pasta app) e http://localhost:8080 abrindo.
- [ ] Saber uma conta **fechada** (ex.: nº 1) e uma **aberta** (ex.: nº 36) para a demo do trigger.
- [ ] Recriar o banco depois de ensaiar (o reajuste e as demos alteram os dados).
- [ ] IDE aberta no AtendimentoRepository e RotinaRepository (P1) e no DashboardRepository e TesteT (P2).
- [ ] Cada pessoa explicou a própria parte em voz alta pelo menos uma vez e leu a parte da outra.

21. Frase de fechamento sugerida

> *"Em resumo: o Java organiza a aplicação em camadas e cuida da interface, mas as regras e quase todos os cálculos ficam no PostgreSQL: constraints, transações, funções, procedimentos, triggers, visões, JOINs, CTEs, funções de janela e estatística. O Java só complementa onde o banco não chega, como o p-valor, e os testes garantem que tudo funciona junto."*